LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASICS WITH COMPOSE

Oleh:

Amandha Citra Mustika NIM. 2410817320004

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Kotlin ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amandha Citra Mustika
NIM : 2410817320004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL PRAKTIKUM 1
A. Source Code 4
B. Output Program 12
C. Pembahasan 15
D. Essay 18
E. Tautan Git 21

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 1
Gambar 2. Tampilan Dadu Setelah Di-Roll 2
Gambar 3. Tampilan Roll Dadu Double 2
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal 12
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung 12
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double 13
Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal 13
Gambar 8. Screenshot Hasil Jawaban Soal XML Belum Beruntung 14
Gambar 9. Screenshot Hasil Jawaban Soal XML Dadu Double 14

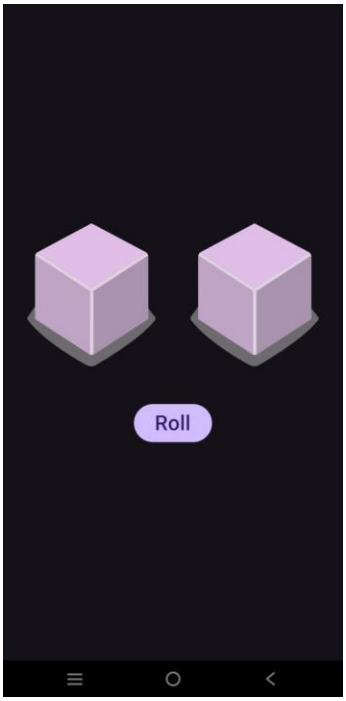
DAFTAR TABEL

Tabel 1. Source Code Main Activity.kt XML 4
Tabel 2. Source Code Main Activity.kt Compose 5
Tabel 3. Source Code activity_main.xml 9

SOAL 1

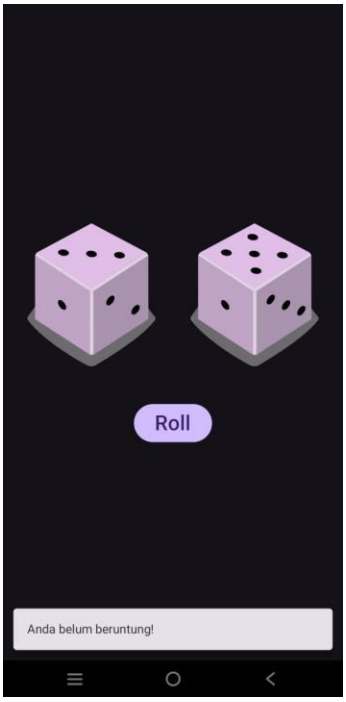
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



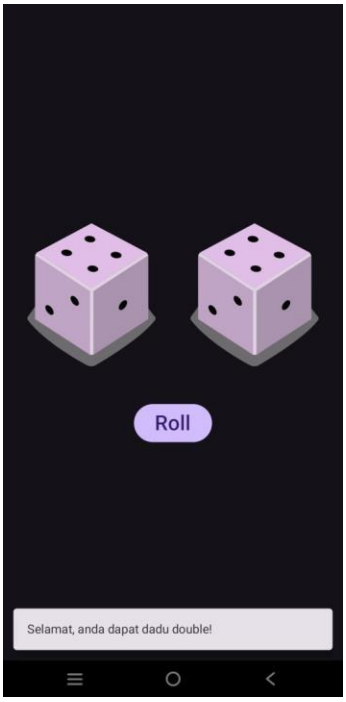
Gambar 1. Tampilan Awal Aplikasi

2. Tampilan Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project.** Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

**** = 2 sumber**

*** = 1 sumber**

Gunakanlah sumber primary artikel/jurnal/buku, web hanya sebagai penunjang opini/argumen

A. Source Code

Tabel 1. Source Code Main Activity.kt XML

No	Main Activity.kt (XML)
1	package com.example.mydiceapplicationxml
2	
3	import android.os.Bundle
4	import android.view.View
5	import android.widget.Button
6	import android.widget.ImageView
7	import android.widget.TextView
8	import androidx.activity.enableEdgeToEdge
9	import androidx.appcompat.app.AppCompatActivity
10	import androidx.core.view.ViewCompat
11	import androidx.core.view.WindowInsetsCompat
12	
13	class MainActivity : AppCompatActivity() {
14	override fun onCreate(savedInstanceState: Bundle?) {
15	super.onCreate(savedInstanceState)
16	*enableEdgeToEdge*()
17	setContentView(R.layout.*activity_main*)
18	val dice1 = findViewById<ImageView>(R.id.*dice1*)
19	val dice2 = findViewById<ImageView>(R.id.*dice2*)
20	val button = findViewById<Button>(R.id.*rollbtn*)
21	val result = findViewById<TextView>(R.id.*resulttxt*)
22	
23	button.setOnClickListener {
24	val roll1 = (1..6).*random*()
25	val roll2 = (1..6).*random*()
26	
27	val imageDice1 = when (roll1) {
28	1 -> R.drawable.*dice_1*
29	2 -> R.drawable.*dice_2*
30	3 -> R.drawable.*dice_3*
31	4 -> R.drawable.*dice_4*
32	5 -> R.drawable.*dice_5*
33	6 -> R.drawable.*dice_6*
34	else -> R.drawable.*dice_0*
35	}
36	
37	val imageDice2 = when (roll2) {
38	1 -> R.drawable.*dice_1*
39	2 -> R.drawable.*dice_2*
40	3 -> R.drawable.*dice_3*
41	4 -> R.drawable.*dice_4*
42	5 -> R.drawable.*dice_5*

```
                6 -> R.drawable.dice_6
                else -> R.drawable.dice_0
            }

            dice1.setImageResource(imageDice1)
            dice2.setImageResource(imageDice2)

            result.visibility = View.VISIBLE

            if (roll1 == roll2) {
                result.text = "Selamat, anda dapat dadu double!"
            } else {
                result.text = "Anda belum beruntung!"
            }
        }

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

Tabel 2. Source Code Main Activity.kt Compose

No	Main Activity.kt (Compose)
1	package com.example.mydiceapplication
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.compose.foundation.Image
8	import androidx.compose.foundation.background
9	import androidx.compose.foundation.layout.Arrangement
10	import androidx.compose.foundation.layout.Box
11	import androidx.compose.foundation.layout.Column
12	import androidx.compose.foundation.layout.Row
13	import androidx.compose.foundation.layout.Spacer
14	import androidx.compose.foundation.layout.fillMaxSize
15	import androidx.compose.foundation.layout.fillMaxWidth

```
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.mydiceapplication.ui.theme.MyDiceApplicationTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyDiceApplicationTheme {
                Scaffold(
                    modifier = Modifier.fillMaxSize(),
                    containerColor = Color.Black
                ) { innerPadding ->
                    DiceRollerApp(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun DiceRollerApp(modifier: Modifier = Modifier) {
    var result1 by remember { mutableIntStateOf(0) }
    var result2 by remember { mutableIntStateOf(0) }

    val imageResource1 = when (result1) {
```

```
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        6 -> R.drawable.dice_6
        else -> R.drawable.dice_0
    }

    val imageResource2 = when (result2) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        6 -> R.drawable.dice_6
        else -> R.drawable.dice_0
    }

    Column(
        modifier = modifier
            .fillMaxSize()
            .background(Color.Black),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Spacer(modifier = Modifier.weight(1f))

        Row(horizontalArrangement = Arrangement.Center) {
            Image(
                painter = painterResource(imageResource1),
                contentDescription = result1.toString()
            )
            Image(
                painter = painterResource(imageResource2),
                contentDescription = result2.toString()
            )
        }

        Button(
            onClick = {
```

```
                result1 = (1..6).random()
                result2 = (1..6).random()
            }
        ) {
            Text(
                text = "Roll",
                fontSize = 20.sp
            )
        }

        Spacer(modifier = Modifier.weight(1f))

        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Bottom
        ) {
            if (result1 != 0 && result2 != 0) {
                if (result1 == result2) {
                    Box(
                        modifier = Modifier
                            .background(color = Color(0xFFE1D9E1))
                            .fillMaxWidth(0.95f)
                            .padding(10.dp)
                    ) {
                        Text(
                            text = "Selamat, anda dapat dadu double!",
                            color = Color.Black
                        )
                    }
                } else {
                    Box(
                        modifier = Modifier
                            .background(color = Color(0xFFE1D9E1))
                            .fillMaxWidth(0.95f)
                            .padding(10.dp)
                    ) {
                        Text(
                            text = "Anda belum beruntung!",
                            color = Color.Black
                        )
                    }
                }
```

```
            }
        }

        Spacer(modifier = Modifier.height(20.dp))
    }
}
```

Tabel 3. Source Code activity_main.xml

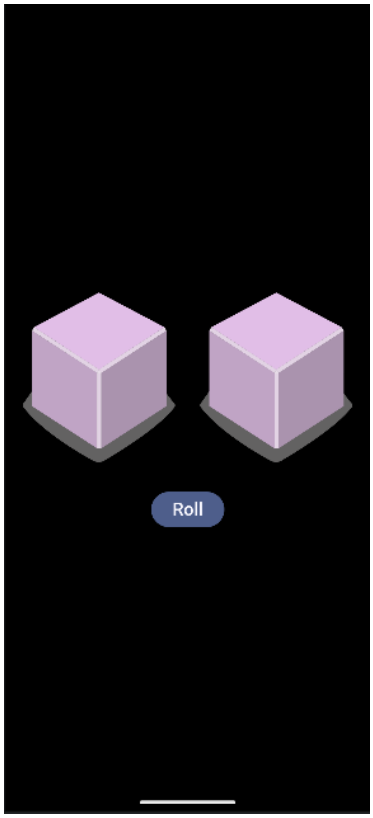
No	activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#000000"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:gravity="fill"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="525dp"
            android:orientation="horizontal"
            android:gravity="center">

            <ImageView
                android:id="@+id/dice1"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:layout_gravity="bottom"
                app:srcCompat="@drawable/dice_0" />

            <ImageView
```

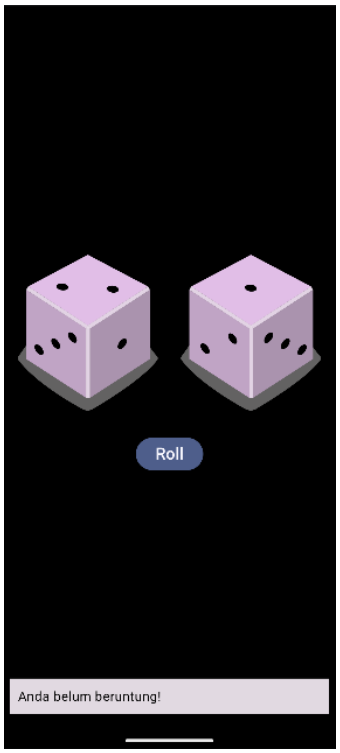
```
                android:id="@+id/dice2"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:layout_gravity="bottom"
                app:srcCompat="@drawable/dice_0" />

        </LinearLayout>

        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:orientation="horizontal">

            <Button
                android:id="@+id/rollbtn"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Roll"
                android:textSize="20sp"
                android:backgroundTint="#d6bfff"
                android:textColor="#000000" />

        </LinearLayout>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:gravity="bottom"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/resulttxt"
                android:layout_width="wrap_content"
                android:layout_height="36dp"
                android:layout_weight="1"
                android:layout_gravity="bottom|center"
                android:layout_marginStart="12dp"
                android:layout_marginEnd="12dp"
                android:layout_marginBottom="12dp"
                android:background="#E1D9E1"
                android:padding="6dp"
                android:textColor="#000000"
                android:visibility="invisible" />

        </LinearLayout>
```

```

    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayout>
```

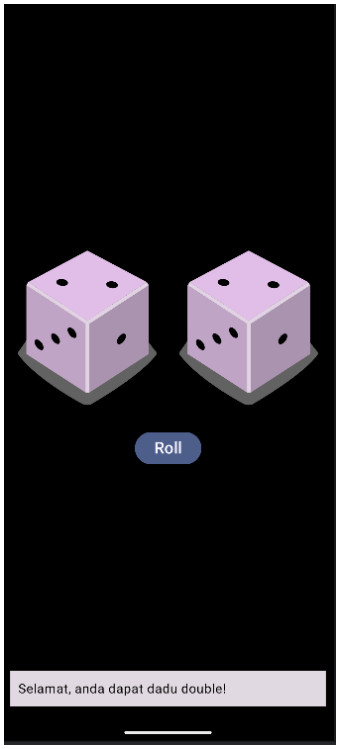
B. Output Program



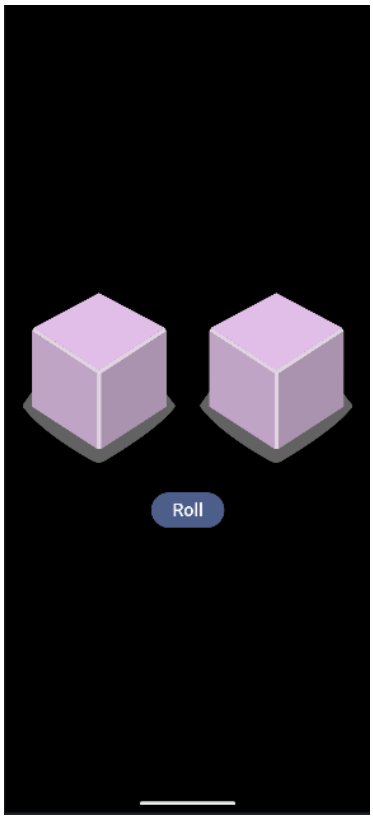
Gambar 4. Screenshot Hasil Jawaban Soal XML Awal



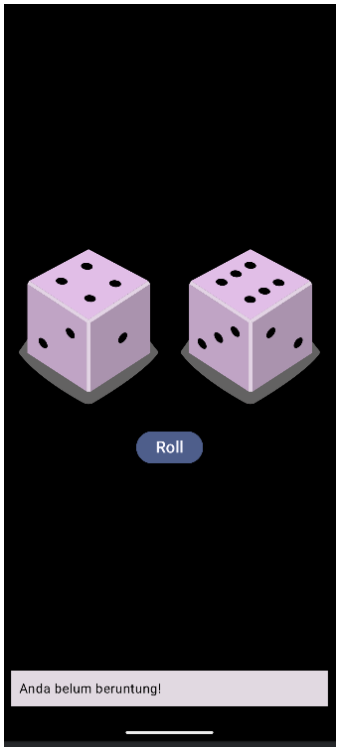
Gambar 5. Screenshot Hasil Jawaban Soal XML Belum Beruntung



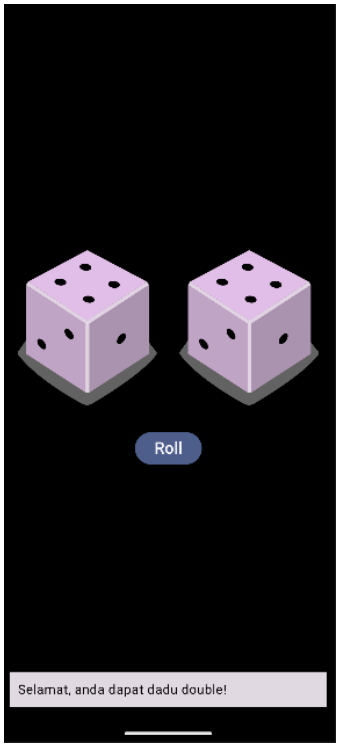
Gambar 6. Screenshot Hasil Jawaban Soal XML Dadu Double



Gambar 7. Screenshot Hasil Jawaban Soal Compose Awal



Gambar 8. Screenshot Hasil Jawaban Soal XML Belum Beruntung



Gambar 9. Screenshot Hasil Jawaban Soal XML Dadu Double

C. Pembahasan

MainActivity.kt XML

File ini berfungsi sebagai entry point aplikasi dadu berbasis XML, di mana logika ditulis di Kotlin sedangkan tampilan diatur di file `activity_main.xml` secara terpisah. Di bagian awal, kode dimulai dengan deklarasi package dan import library yang dibutuhkan, seperti `ImageView`, `Button`, `TextView`, dan `library` pendukung lainnya. Class MainActivity mewarisi AppCompatActivity, yang memberikan akses ke lifecycle method seperti onCreate. Method onCreate sendiri adalah method pertama yang dipanggil saat aplikasi dibuka, dan di sinilah semua setup awal dilakukan.

Di dalam onCreate, pertama dipanggil `enableEdgeToEdge()` agar tampilan aplikasi bisa memenuhi seluruh layar termasuk area status bar. Kemudian `setContentView` menghubungkan file layout activity_main.xml ke Activity ini. Setelah itu, setiap komponen UI seperti dice1, dice2, button, dan result diambil menggunakan `findViewById` agar bisa dikontrol dari kode Kotlin.

Selanjutnya, button.`setOnClickListener` digunakan untuk mendeteksi ketika user menekan tombol Roll. Di dalamnya, roll1 dan roll2 masing-masing menghasilkan angka acak antara 1 sampai 6 menggunakan `(1..6).random()`. Angka tersebut kemudian dicocokkan ke gambar dadu yang sesuai menggunakan blok when, yang hasilnya disimpan ke `imageDice1` dan `imageDice2`. Gambar lalu ditampilkan ke ImageView menggunakan `setImageResource`.

Setelah gambar diperbarui, result.visibility diubah menjadi visibel agar pesan hasil muncul. Jika roll1 sama dengan roll2 maka ditampilkan pesan "Selamat, anda dapat dadu double!", sedangkan jika berbeda ditampilkan "Anda belum beruntung!".Terakhir, `ViewCompat.setOnApplyWindowInsetsListener` digunakan untuk mengatur padding secara otomatis agar konten tidak tertutup oleh system bar seperti status bar dan navigation bar.

activity_main.xml

Layout XML ini berfungsi untuk mendefinisikan tampilan visual dari layar utama aplikasi, berisi dua gambar dadu, tombol roll, dan teks hasil sebagai bagian UI dari

MainActivity.kt. Dimulai dengan deklarasi <?xml version="1.0" encoding="utf-8"?> yang menandakan file XML dengan format karakter UTF-8. Bagian pertama adalah <androidx.constraintlayout.widget.ConstraintLayout> sebagai root layout yang menampung seluruh komponen UI, dengan match_parent pada width dan height agar memenuhi seluruh layar, android:id="@+id/main" agar bisa diakses dari Kotlin, dan android:background="#000000" untuk mengatur warna latar menjadi hitam. Kemudian terdapat <LinearLayout> pertama sebagai container utama dengan orientation="vertical" sehingga isi disusun dari atas ke bawah, dan constraint ke semua sisi parent agar mengisi seluruh area ConstraintLayout.

Di dalamnya terdapat <LinearLayout> kedua khusus untuk area dadu, dengan orientation="horizontal" agar dua dadu berdampingan, layout_height="525dp" sebagai tinggi tetap, dan gravity="center" agar isi berada di tengah. Di dalam LinearLayout ini terdapat dua <ImageView> yaitu dice1 dan dice2, masing-masing dengan layout_weight="1" agar ruang terbagi merata, layout_gravity="bottom" agar gambar berada di bawah container, dan app:srcCompat="@drawable/dice_0" sebagai gambar awal saat aplikasi pertama dibuka.

Selanjutnya terdapat <LinearLayout> ketiga sebagai wadah tombol, dengan wrap_content pada width dan height serta layout_gravity="center" agar tombol berada di tengah layar tepat di bawah dadu. Di dalamnya terdapat <Button> dengan android:id="@+id/rollbtn", android:text="Roll" sebagai label, android:textSize="20sp" sebagai ukuran teks, android:backgroundTint="#d6bfff" untuk warna ungu muda pada tombol, dan android:textColor="#000000" agar teks tombol berwarna hitam.

Terakhir, <LinearLayout> keempat sebagai wadah hasil dengan gravity="bottom" agar isi menempel ke bawah layar. Di dalamnya terdapat <TextView> dengan android:id="@+id/resulttxt", layout_height="36dp" sebagai tinggi tetap, layout_weight="1" agar

memenuhi lebar layar, `layout_gravity="bottom|center"` agar posisinya di bawah tengah, margin kiri kanan dan bawah masing-masing 12dp, `android:background="#E1D9E1"` sebagai warna latar abu-abu, android:padding="6dp" untuk jarak dalam, android:textColor="#000000" agar teks berwarna hitam, dan `android:visibility="invisible"` agar tidak terlihat saat aplikasi pertama dibuka sebelum tombol Roll ditekan.

MainActivity.kt Compose

File ini merupakan implementasi aplikasi lempar dadu menggunakan pendekatan modern Android yaitu Jetpack Compose. Berbeda dengan versi XML sebelumnya, seluruh tampilan dan logika UI ditulis langsung dalam Kotlin menggunakan fungsi `@Composable` tanpa memerlukan file layout terpisah. Aplikasi ini menghasilkan dua angka acak antara 1 sampai 6, menampilkan gambar dadu sesuai angka tersebut, lalu menampilkan pesan "double" jika kedua angka sama atau "belum beruntung" jika berbeda.

Di bagian awal, MainActivity berjalan sebagai entry point aplikasi. Di dalam onCreate, dipanggil `enableEdgeToEdge()` agar UI memenuhi seluruh layar, kemudian `setContent` dipanggil untuk memulai rendering UI menggunakan Compose, yang berarti tidak ada lagi file XML yang digunakan untuk mengatur tampilan. UI kemudian dibungkus dalam `MyDiceApplicationTheme` dan `Scaffold`. Di sini `containerColor` diatur menjadi `Color.Black` sehingga seluruh latar layar berwarna hitam. Scaffold sendiri berfungsi sebagai kerangka dasar layar yang memberikan `innerPadding` agar konten tidak tertutup system bar.

Selanjutnya fungsi `DiceRollerApp` dipanggil dengan menerima modifier dari innerPadding. Di dalam fungsi ini, dua variabel state yaitu result1 dan result2 dibuat menggunakan remember dan `mutableIntStateOf(0)`. Nilai awal 0 berarti dadu belum dilempar sama sekali. Setelah state dibuat, `imageResource1` dan `imageResource2` ditentukan menggunakan blok when yang mencocokkan nilai result dengan gambar dadu yang sesuai. Jika nilai 0 atau di luar 1 sampai 6 maka akan menampilkan `dice_0` sebagai gambar dadu kosong.

Masuk ke bagian UI utama, Column digunakan untuk menyusun elemen secara vertikal dengan background hitam dan posisi konten di tengah. Spacer dengan `weight(1f)` diletakkan di atas dan bawah Row agar dadu dan tombol berada di tengah layar secara vertikal. Di dalam Row, dua Image ditampilkan berdampingan menggunakan `painterResource` yang mengambil gambar sesuai nilai `imageResource1` dan `imageResource2`. Kemudian terdapat Button dengan `onClick` yang akan mengubah result1 dan result2 menjadi angka acak baru menggunakan `(1..6).random()` setiap kali tombol Roll ditekan. Karena Compose bersifat reaktif, perubahan state ini otomatis menyebabkan seluruh UI dirender ulang dengan gambar dadu yang baru.

Di bagian bawah, terdapat Column kedua yang berisi pengecekan kondisi. Jika result1 dan result2 keduanya tidak sama dengan 0, artinya tombol sudah pernah ditekan, maka pesan hasil akan ditampilkan. Jika result1 sama dengan result2 maka Box dengan background `Color(0xFFE1D9E1)` menampilkan teks "Selamat, anda dapat dadu double!", sedangkan jika berbeda menampilkan "Anda belum beruntung!". Box di sini berfungsi seperti container dengan lebar 95% layar dan padding 10dp di dalamnya.

Terakhir, `Spacer` dengan tinggi 20dp diletakkan paling bawah untuk memberikan jarak antara pesan hasil dengan tepi bawah layar.

D. Essay

1. Riset dan cite artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **

   Jawab:

   Dalam pengembangan perangkat lunak, terdapat perbedaan mendasar antara paradigma imperative dan declarative. Michael L. Scott menjelaskan bahwa pemrograman imperative berfokus pada urutan instruksi yang mengubah status program langkah demi langkah, di mana pemrogram harus menentukan secara eksplisit "bagaimana" suatu hasil dicapai [1]. Sebaliknya, paradigma declarative lebih berfokus pada deskripsi logika komputasi tanpa merinci alur kontrolnya, atau dengan kata lain menekankan pada "apa" yang ingin ditampilkan [1].

Penerapannya dalam pembuatan antarmuka pengguna (UI) terlihat jelas pada evolusi framework modern. Dalam pendekatan imperative (seperti Java Swing atau DOM manipulation manual), pengembang harus menulis kode untuk memperbarui UI secara manual setiap kali data berubah. Namun, pada pendekatan declarative seperti React, UI dipandang sebagai fungsi dari state. Pengembang cukup mendeskripsikan bagaimana tampilan UI untuk setiap status (state) yang mungkin terjadi, dan framework akan menangani transisi serta pembaruan elemen secara otomatis [3].

Opini Pribadi:

Berdasarkan riset tersebut, paradigma declarative jauh lebih unggul untuk pembuatan UI modern. Hal ini dikarenakan kode menjadi lebih ringkas, mudah diprediksi, dan meminimalisir bug yang sering muncul akibat sinkronisasi state manual yang kompleks pada pendekatan imperative.

2. Apa yang membedakan OOP pada kotlin dengan OOP pada java? *

Jawab:

Perbedaan utama antara OOP di Kotlin dan Java terletak pada filosofi desain yang lebih ringkas dan aman. Menurut Sebastian Aigner, salah satu perbedaan paling signifikan adalah bahwa di Kotlin, semua class dan method bersifat final secara default [1]. Ini berbeda dengan Java yang secara default bersifat open, sehingga di Kotlin kita harus secara eksplisit menggunakan kata kunci open jika ingin sebuah class dapat diwarisi.

Selain itu, Kotlin memperkenalkan konsep Properties yang menggantikan Fields di Java. Di Java, kita sering kali harus menulis getter dan setter secara manual, sedangkan di Kotlin, deklarasi properti secara otomatis menghasilkan aksesori tersebut, sehingga mengurangi boilerplate code yang tidak perlu [2]. Kotlin juga memiliki sistem keamanan null (null-safety) yang terintegrasi langsung dalam sistem tipenya, sesuatu yang tidak tersedia secara native di OOP Java tradisional.

3. Sebutkan, dan jelaskan mengenai SOLID Principles *

   Jawab:

   Prinsip SOLID dalam arsitektur perangkat lunak modern dirancang untuk menciptakan sistem yang mudah dikelola dan siap menghadapi perubahan:

- **Single Responsibility Principle (SRP)**

  Menekankan bahwa sebuah kelas tidak hanya harus melakukan satu hal, tetapi harus memiliki "satu alasan untuk berubah". Alasan ini biasanya dikaitkan dengan satu kelompok pengguna atau aktor tertentu. Hal ini mencegah dampak samping (side effects) yang tidak terduga saat satu bagian kode diubah untuk kepentingan satu departemen, namun tanpa sengaja merusak fungsi bagi departemen lain.

- **Open/Closed Principle (OCP)**

  Konsep ini mengarahkan pengembang untuk merancang sistem yang bisa diperluas perilakunya tanpa harus memodifikasi kode sumber yang sudah ada. Pendekatan ini biasanya dilakukan dengan menggunakan abstraksi atau interface, sehingga fitur baru bisa ditambahkan sebagai "plugin" ke dalam sistem tanpa menyentuh kode inti yang sudah stabil.

- **Liskov Substitution Principle (LSP)**

  Prinsip ini menyatakan bahwa objek dari subtipe harus benar-benar dapat menggantikan objek dari supertipe tanpa mengubah kebenaran program. Jika sebuah kelas turunan tidak bisa menggantikan fungsi kelas induknya secara identik dalam hal perilaku, maka desain tersebut melanggar hierarki objek yang sehat.

- **Interface Segregation Principle (ISP)**

  Fokus pada pencegahan ketergantungan yang tidak perlu. Pengembang harus menghindari pembuatan interface yang "gemuk" (memiliki terlalu banyak metode). Lebih baik memecah interface besar menjadi beberapa interface kecil yang spesifik, sehingga klien hanya perlu mengetahui metode yang benar-benar mereka gunakan.

- **Dependency Inversion Principle (DIP)**

  Prinsip ini membalik arah ketergantungan tradisional. Komponen tingkat tinggi (logika bisnis) tidak boleh bergantung pada komponen tingkat rendah (database atau UI). Sebaliknya, keduanya harus bergantung pada abstraksi (interface). Hal ini membuat logika bisnis tetap murni dan tidak terpengaruh oleh detail teknis infrastruktur [4].

4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

   Jawab:

   Perbedaan utama antara class biasa dan data class di Kotlin terletak pada tujuan penggunaannya dan efisiensi penulisan kode. Sebuah class biasa umumnya kita gunakan sebagai cetak biru untuk objek yang kompleks, di mana objek tersebut memiliki logika, status, dan behavior tertentu. Jika kita ingin membandingkan isi dari dua objek class biasa atau ingin mencetak isinya agar mudah dibaca, kita harus menulis fungsi `equals(), hashCode(), dan toString()` secara manual. Sebaliknya, data class diciptakan secara khusus hanya untuk menyimpan atau membawa data seperti Data Transfer Object atau Model. Dengan hanya menambahkan kata kunci data di depan kelas, Kotlin sangat memanjakan kita karena sistem secara otomatis membuatkan fungsi equals(), hashCode(), dan toString() berdasarkan properti yang ada di dalam constructor utamanya. Selain itu, data class juga otomatis menyediakan fungsi copy() yang sangat praktis untuk menduplikasi objek sambil mengubah nilai properti tertentu, serta memungkinkan kita membongkar isi objek langsung ke dalam variabel-variabel terpisah. Intinya, data class membuat kode kita jauh lebih bersih dan bebas dari boilerplate saat kita hanya butuh tempat untuk menampung data.

E. Tautan Git

Berikut adalah tautan untuk repository yang dibuat untuk source code:

https://github.com/zvyaa/MODUL-MOBILE

F. Daftar Pustaka

[1] Michael L. Scott, “In Praise of Programming Language Pragmatics,Third Edition,” 2009. doi: https://doi.org/10.1016/B978-0-12-374514-9.X0001-8.

[2] R. E. S. I. D. J. Sebastian Aigner, “Kotlin in Action, Second Edition MEAP V09,” 2024.

[3] React, “Reacting to Input with State.” Accessed: Apr. 03, 2026. [Online]. Available: https://react.dev/learn/reacting-to-input-with-state

[4] R. C. Martin, “Clean Architecture: A Craftsman’s Guide to Software Structure and Design,” 2018.